U N E W E B

NIVEL 8

Desarrollo de Aplicaciones de IA y Wrappers No-Code

Manual teórico-práctico con Google Gemini API

20 horas académicas | 2 módulos | 5 prácticas

Plataformas: Bubble, Glide, FlutterFlow
Modelo de IA: Google Gemini (API gratuita de Google AI Studio)
Pagos: Stripe (modo de prueba)

Octubre 2026

Contenido

Presentación del nivel

Objetivo, competencias, estructura de horas, requisitos y evaluación

Antes de comenzar: preparación y vigencia

Por qué Gemini, avisos de vigencia y lista de verificación del primer día

MÓDULO 8.1: Plataformas No-Code para Apps de IA (10 h)

Unidad 1. Panorama de plataformas no-code y low-code

Unidad 2. Wrappers de IA: envolver un modelo en una interfaz

Unidad 3. Diseño de interfaces conversacionales y formularios inteligentes

Unidad 4. Conexión de un builder no-code a la API de Gemini

Unidad 5. Gestión de usuarios, autenticación y planes de suscripción

Práctica P1. Prototipo de app de IA en una plataforma no-code

Práctica P2. Conexión de la app a la API de Gemini

Práctica P3. Integración de un sistema de autenticación de usuarios

MÓDULO 8.2: Despliegue y Monetización de Apps de IA (10 h)

Unidad 6. Integración de pagos (Stripe) en una app de IA no-code

Unidad 7. Despliegue y hosting de aplicaciones no-code

Unidad 8. Pruebas de usabilidad y mejora iterativa

Unidad 9. Métricas de uso y analítica de producto

Unidad 10. Estrategias de monetización: SaaS, freemium y créditos

Práctica P4. Integración de pagos con Stripe en la aplicación

Práctica P5 (Proyecto final). Wrapper de IA funcional, desplegado y monetizable

Anexos

A. Plantillas de instrucción del sistema · B. Banco de pruebas · C. Solución de problemas · D. Glosario · E. Recursos y fuentes oficiales

Presentación del nivel

Objetivo general

Construir y desplegar aplicaciones funcionales potenciadas por IA **sin escribir código tradicional**, usando plataformas no-code y wrappers que envuelven un modelo de lenguaje (Google Gemini) en una interfaz de usuario con autenticación, pagos y analítica.

Competencias que desarrollará el participante

- Seleccionar la plataforma no-code adecuada (Bubble, Glide o FlutterFlow) según el tipo de producto.
- Diseñar un wrapper de IA: interfaz, instrucciones del sistema y flujo de datos.
- Conectar un builder no-code a la API de Gemini de forma segura.
- Gestionar usuarios, autenticación, privacidad de datos y planes.
- Integrar pagos con Stripe y actualizar planes o créditos de forma automática.
- Desplegar la aplicación, medir su uso y mejorarla con usuarios reales.
- Calcular costos de IA y definir un modelo de monetización sostenible.

Estructura y distribución de horas

Módulo	Unidades	Práctica	Horas
8.1 Plataformas No-Code para Apps de IA	U1 Panorama no-code · U2 Wrappers de IA · U3 Interfaces conversacionales · U4 Conexión a Gemini · U5 Usuarios y planes	P1, P2, P3	10 h
8.2 Despliegue y Monetización	U6 Pagos con Stripe · U7 Despliegue y hosting · U8 Usabilidad e iteración · U9 Métricas · U10 Monetización	P4, P5	10 h

Actividad	Horas sugeridas
Teoría guiada (clase expositiva con demostración)	8 h
Prácticas guiadas P1 a P4 (≈ 2,5 h cada una)	10 h
Proyecto final P5 (desarrollo y presentación)	2 h en clase + trabajo autónomo

Requisitos previos

- Manejo básico de internet, hojas de cálculo y bases de datos conceptuales (tablas, campos, relaciones).
- Haber cursado los niveles de automatización (Zapier, Make, n8n) es recomendable, pero no obligatorio.

- Cuenta de Google (para Google AI Studio), cuenta en la plataforma no-code elegida y cuenta de Stripe en modo de prueba.

Evaluación del nivel

Componente	Peso
P1 Prototipo de app de IA	15 %
P2 Conexión a la API de Gemini	15 %
P3 Autenticación de usuarios	15 %
P4 Pagos con Stripe	20 %
P5 Proyecto final: wrapper desplegado y monetizable	35 %

Antes de comenzar: preparación y vigencia

Por qué Gemini en este curso

Google ofrece una API gratuita para modelos de la familia Flash, accesible con solo una cuenta de Google. Esto permite que cada participante construya y pruebe su aplicación completa sin tarjeta de crédito. Las prácticas P1 a P4 pueden realizarse íntegramente con el plan gratuito.

Lo que cambia con frecuencia (verificar siempre)

Aviso de vigencia

Nombres de modelos: Google renueva su catálogo con frecuencia. Los modelos de la familia 2.5 tienen fecha de cierre anunciada para el 16 de octubre de 2026, por lo que este manual usa la marca `MODEL_ID` y se debe copiar el nombre del modelo Flash vigente desde Google AI Studio.

Límites gratuitos: Google ya no publica una tabla fija; los límites son por proyecto y se consultan en la consola de AI Studio (aistudio.google.com/rate-limit). Cada participante debe anotar los suyos en la P2.

Privacidad y uso comercial: en el plan gratuito, Google puede usar los datos enviados para mejorar sus productos y se debe revisar si los términos permiten uso comercial. No se deben enviar datos reales de clientes. Para la P5 con usuarios reales, se debe activar la facturación (nivel de pago).

Planes de las plataformas: las funciones disponibles en los planes gratuitos de Bubble, Glide y FlutterFlow cambian. Verificar antes de cada cohorte qué funciones (API, flujos de backend, dominio propio) requieren plan de pago.

Lista de verificación del primer día

1. Crear la clave de API en Google AI Studio (aistudio.google.com → **Get API key**).
2. Anotar el nombre del modelo Flash vigente y los límites del proyecto.
3. Crear la cuenta en la plataforma no-code elegida (Bubble recomendado para web).
4. Crear una cuenta de Stripe y confirmar que está en **modo de prueba**.
5. Crear una carpeta de trabajo con: capturas, documento del prompt y bitácora de pruebas.

MÓDULO 8.1: Plataformas No-Code para Apps de IA

Duración: 10 horas | **Unidades:** 1 a 5 | **Prácticas:** P1, P2 y P3

Unidad 1. Panorama de plataformas no-code y low-code

1.1 Concepto

No-code significa construir software mediante interfaces visuales, sin escribir código. **Low-code** admite fragmentos de código para casos especiales. En una app de IA, lo que se construye visualmente es la interfaz, la base de datos, la lógica de negocio y la llamada a la API del modelo.

1.2 Las tres plataformas del curso

Criterio	Bubble	Glide	FlutterFlow
Tipo de producto	Aplicaciones web completas (SaaS)	Apps internas y móviles sobre tablas	Apps móviles nativas (iOS/Android) y web
Fortaleza	Base de datos, lógica y conector de API integrados	Velocidad: prototipo en horas	Diseño visual avanzado; exportación de código
Conexión a APIs	API Connector (muy flexible)	Acciones de llamada a API y complementos según el plan	API Calls configurables en el builder
Backend propio	Sí, incluido	Tablas propias o hojas de cálculo	Requiere Firebase o Supabase
Curva de aprendizaje	Media-alta	Baja	Media-alta
Limitación típica	Rendimiento si el diseño de datos es deficiente	Menor control de lógica compleja	Exige comprender conceptos de backend
Ideal para	Wrapper de IA con usuarios y pagos	Herramienta interna de un equipo	App móvil publicada en tiendas

1.3 ¿Cómo elegir?

1. **¿Web o móvil?** Para un SaaS que se vende por navegador, Bubble; para una app instalable, FlutterFlow.
2. **¿Necesita lógica propia?** Si hay créditos, planes, webhooks y reglas de privacidad, se requiere un backend real (Bubble).
3. **¿Cuánto tiempo hay?** Para validar una idea en un día, Glide.
4. **¿Quién será dueño de los datos?** Se debe considerar la exportabilidad y la dependencia de la plataforma (*vendor lock-in*).

Decisión del curso

Los ejemplos detallados de este manual usan **Bubble** porque cubre en una sola herramienta la interfaz, la base de datos, la autenticación, el API Connector y los flujos de backend. Cada práctica indica la alternativa para **Glide** y **FlutterFlow**.

1.4 Riesgos de las plataformas no-code

- **Dependencia del proveedor:** migrar fuera de la plataforma suele implicar reconstruir.
- **Costos escalables:** el plan de la plataforma crece con el uso, además del costo de la API de IA.
- **Límites de rendimiento:** consultas mal diseñadas ralentizan la app.
- **Seguridad por configuración:** un error de reglas de privacidad expone datos de usuarios.

Unidad 2. Wrappers de IA: envolver un modelo en una interfaz

2.1 Definición

Un **wrapper (envoltura) de IA** es una aplicación que usa un modelo de lenguaje existente (aquí, Gemini) y le añade una interfaz diseñada, instrucciones especializadas y datos propios para resolver un problema concreto. El usuario no necesita saber escribir prompts: la aplicación lo hace por él.

2.2 Anatomía de un wrapper

Capa	Qué contiene	Dónde se construye en no-code
Interfaz	Formularios, botones, pantalla de resultados, historial	Editor visual de páginas
Instrucciones (cerebro)	Instrucción del sistema (`systemInstruction`) con rol, tono, formato y reglas	Campo de texto o tabla de configuración
Lógica de negocio	Validaciones, límites de uso, créditos, planes	Flujos de trabajo (workflows)
Datos	Usuarios, generaciones, planes, plantillas	Base de datos del builder
Modelo de IA	Gemini, invocado por HTTP	API Connector / acción de llamada a API

2.3 Flujo de datos

Flujo de un wrapper

```
Usuario
  → completa un formulario (tono, público, tema)
  → el workflow valida entrada y verifica créditos o plan
  → arma el prompt con la plantilla + datos del formulario
  → llama a la API de Gemini (clave privada en el servidor)
  → recibe el texto generado
  → guarda la generación en la base de datos (usuario, entrada, salida, fecha)
  → descuenta crédito y muestra el resultado
```

2.4 Dónde está el valor

Cualquiera puede abrir un chat con un modelo. Un wrapper vale cuando entrega **un resultado listo para usar en un contexto específico**:

- **Nicho:** un público concreto (docentes, abogados, vendedores, restaurantes).
- **Formato:** el resultado sale estructurado (correo con asunto, tabla, plan de clase).
- **Contexto guardado:** el usuario no repite sus datos cada vez.
- **Flujo completo:** generar, editar, guardar, exportar y compartir.

2.5 Ejemplos de wrappers rentables

Wrapper	Entrada del usuario	Salida
Generador de propuestas comerciales	Cliente, servicio, presupuesto, plazo	Propuesta formal de 1 página
Respuestas a reseñas de negocios	Texto de la reseña, calificación, tono	Respuesta lista para publicar
Planes de clase	Tema, nivel, duración, objetivos	Plan estructurado con actividades
Descripciones de producto	Nombre, atributos, público	Texto SEO y bullets de beneficios
Resumidor de contratos	Texto del contrato (pegado)	Resumen y puntos de atención

> **Advertencia de responsabilidad**
> Un wrapper que genera contenido legal, médico o financiero debe mostrar un aviso claro de que el resultado **no sustituye a un profesional**, y no debe presentarse como asesoría definitiva.

Unidad 3. Diseño de interfaces conversacionales y formularios inteligentes

3.1 Tres patrones de interfaz

Patrón	Cuándo usarlo	Ventaja	Riesgo
Chat libre	Soporte, exploración, preguntas abiertas	Flexible, natural	Prompts impredecibles; consumo alto de cuota
Formulario guiado	Tareas repetibles con resultado definido	Resultados consistentes; menor costo	Menos flexible
Híbrido	Resultado inicial + ajustes	Lo mejor de ambos	Mayor complejidad de diseño

Para un wrapper con cuota gratuita limitada, el **formulario guiado** suele ser la mejor opción: controla la entrada, reduce errores y consume menos peticiones.

3.2 Formularios inteligentes

Un formulario inteligente convierte campos simples en un prompt bien construido mediante una **plantilla con variables**:

Plantilla de prompt con variables

```
Redacta una propuesta comercial en español.
Cliente: {{cliente}}
Servicio ofrecido: {{servicio}}
Presupuesto: {{presupuesto}}
Plazo de entrega: {{plazo}}
Tono: {{tono}}

Estructura obligatoria: 1) Resumen, 2) Alcance, 3) Inversión, 4) Siguientes pasos.
Máximo 300 palabras.
```

En el builder, cada `{{variable}}` se reemplaza por el valor del campo correspondiente (en Bubble, mediante *Insert dynamic data*). Los **campos de selección** (tono, idioma, longitud) reducen la ambigüedad frente a los de texto libre.

3.3 Buenas prácticas de experiencia de usuario

- **Estado de carga:** mostrar “Generando…” y deshabilitar el botón para evitar peticiones duplicadas.
- **Límites de entrada:** contador de caracteres y validación de campos obligatorios.
- **Acciones sobre el resultado:** copiar, regenerar, editar y guardar.
- **Ejemplos precargados:** un botón “Probar con un ejemplo” reduce la fricción inicial.
- **Mensajes de error comprensibles:** “Hemos alcanzado el límite de uso por hoy, intenta en unos minutos”, en lugar de un error técnico.
- **Transparencia:** indicar que el contenido fue generado por IA y que debe revisarse.

3.4 Diseño de la instrucción del sistema

La instrucción del sistema fija el comportamiento del wrapper. Una estructura recomendada:

Bloque	Pregunta que responde	Ejemplo
Rol	¿Quién es el asistente?	Eres un redactor comercial senior para pymes de Latinoamérica.
Tarea	¿Qué debe producir?	Redacta propuestas comerciales a partir de los datos recibidos.
Formato	¿Cómo debe verse la salida?	Usa títulos en negrita y no más de 300 palabras.
Tono	¿Con qué voz?	Profesional, cercano y sin jerga innecesaria.
Límites	¿Qué no debe hacer?	No inventes cifras ni datos del cliente; si falta información, indícalo.

Regla de oro del prompt

Un buen prompt de wrapper es específico, estructurado y con límites. Se prueba con al menos 10 entradas distintas (incluyendo entradas vacías, extrañas y maliciosas) antes de publicarlo.

Unidad 4. Conexión de un builder no-code a la API de Gemini

4.1 Obtener la clave de API

1. Ingresar a **aistudio.google.com** con una cuenta de Google.
2. Pulsar **Get API key** y crear una clave asociada a un proyecto.
3. Copiar la clave y guardarla en un lugar seguro (nunca en capturas de pantalla, correos ni repositorios públicos).
4. Abrir la consola de límites (**aistudio.google.com/rate-limit**) y anotar el modelo Flash disponible y sus cuotas.

4.2 Anatomía de una petición

Elemento	Valor
Método	POST
URL	`https://generativelanguage.googleapis.com/v1beta/models/MODEL_ID:generateContent`
Header de autenticación	`x-goog-api-key: TU_CLAVE`
Header de contenido	`Content-Type: application/json`
Cuerpo	JSON con `systemInstruction`, `contents` y `generationConfig`

Cuerpo de la petición

```
{
  "systemInstruction": {
    "parts": [{ "text": "Eres un redactor comercial senior para pymes. Responde en español,
máximo 300 palabras." }]
  },
  "contents": [
    { "role": "user",
      "parts": [{ "text": "Cliente: Panadería Luna. Servicio: rediseño de logo.
Presupuesto: 400 USD." }] }
  ],
  "generationConfig": {
    "temperature": 0.7,
    "maxOutputTokens": 1000
  }
}
```

Estructura de la respuesta (resumida)

```
{
  "candidates": [
    {
      "content": { "role": "model", "parts": [{ "text": "Propuesta comercial: ..." }] },
      "finishReason": "STOP"
    }
  ],
  "usageMetadata": { "promptTokenCount": 85, "candidatesTokenCount": 210,
"totalTokenCount": 295 }
}
```

El texto generado está en `candidates[0].content.parts[0].text`. El objeto `usageMetadata` informa los tokens consumidos y es la base para calcular el costo por generación (Unidad 10).

4.3 Parámetros que controla el diseñador

Parámetro	Función	Recomendación para wrappers
`temperature`	Creatividad (0 = más predecible, valores altos = más variado)	0.3 a 0.5 para resúmenes y datos; 0.7 a 0.9 para textos creativos
`maxOutputTokens`	Tope de longitud de la respuesta	Fijarlo según el formato; limita el costo
`systemInstruction`	Comportamiento fijo del wrapper	Ver estructura de la Unidad 3.4
`responseMimeType`	Formato de salida (por ejemplo, `application/json`)	Útil cuando la app debe leer campos separados

Respuestas cortadas o vacías

Los modelos con razonamiento pueden consumir parte de `maxOutputTokens` en su “pensamiento” interno. Si `finishReason` es `MAX_TOKENS` o el texto sale incompleto, se debe aumentar `maxOutputTokens` o ajustar la configuración de razonamiento según la documentación vigente del modelo.

4.4 Salida estructurada (JSON)

Cuando la app debe mostrar varios campos (asunto, cuerpo, resumen), conviene pedir la salida en JSON y leer cada campo por separado. Se indica `responseMimeType` como `application/json` y se describe la estructura esperada (con `responseSchema` o dentro de la instrucción). Esto evita analizar texto libre y hace la interfaz más robusta.

Ejemplo de configuración

```
"generationConfig": {
  "responseMimeType": "application/json"
}
// En la instrucción del sistema:
// Devuelve únicamente un JSON con las claves: "asunto", "cuerpo", "resumen".
```

4.5 Conversaciones de varios turnos

La API **no recuerda** conversaciones previas. Para un chat, la app debe reenviar el historial en cada petición como una lista alternada de mensajes con rol `user` y rol `model`. En no-code, el historial se reconstruye desde la tabla de mensajes del usuario. Para controlar costos, se envía solo los últimos N mensajes (por ejemplo, 10).

4.6 Códigos de error frecuentes

Código	Significado	Acción en la app
400	Petición mal formada (JSON inválido, parámetro incorrecto)	Revisar el cuerpo; usar "formatted as JSON-safe" en las variables
403	Clave inválida o sin permiso	Verificar la clave y que el header sea `x-goog-api-key`
404	Modelo no encontrado o retirado	Actualizar `MODEL_ID` desde AI Studio
429	Límite de cuota alcanzado (RESOURCE_EXHAUSTED)	Mostrar mensaje amable, esperar y reintentar
500 / 503	Error temporal del servicio	Reintentar con espera breve
Sin `candidates`	Contenido bloqueado por filtros de seguridad	Pedir al usuario reformular la entrada

4.7 Seguridad de la clave

- La clave debe configurarse como **parámetro privado** del conector, de modo que la llamada se ejecute en el servidor del builder y la clave nunca llegue al navegador.
- No pegar la clave dentro de elementos visibles de la página, campos de texto ni plantillas.
- Si la clave se filtra, **revocarla** en AI Studio y crear una nueva.
- Usar una clave distinta para pruebas y para producción.

4.8 Diseñar la app para cuotas bajas

Con el plan gratuito, todas las peticiones de todos los usuarios comparten el cupo del proyecto. Medidas de diseño:

- **Límite diario por usuario** (`usos_hoy`) para que un solo usuario no agote el cupo.
- **Un clic, una petición:** deshabilitar el botón mientras se genera.
- **Caché:** si la entrada es idéntica a una anterior, reutilizar el resultado guardado.
- **Entradas acotadas:** límite de caracteres para controlar tokens.
- **Reintento con espera** ante el error 429.

4.9 Configuración en Bubble (API Connector)

1. En **Plugins**, instalar **API Connector**.
2. Crear una nueva API con el nombre `Gemini`. En autenticación, dejar "None".

3. Agregar el header compartido `x-goog-api-key` con la clave y marcar **Private**.
4. Agregar una llamada `generar`: método **POST**, **Use as: Action**, URL con el `MODEL_ID` vigente, cuerpo JSON.
5. En el cuerpo, usar parámetros entre ángulos: `<sistema>` y `<entrada>`, y marcar los textos libres con el formato JSON-safe.
6. Activar la opción para **incluir errores en la respuesta y permitir que el workflow continúe**, de modo que la app pueda mostrar un mensaje propio.
7. Pulsar **Initialize call** con valores de ejemplo para que Bubble detecte la estructura de la respuesta.

Plataforma	Cómo se conecta
Bubble	API Connector con llamada POST como acción (descrito arriba).
Glide	Acción o complemento de llamada a API, o un puente mediante Make/n8n/webhook si el plan no permite llamadas directas.
FlutterFlow	API Calls en el builder; para ocultar la clave, preferible un backend intermedio (función de Firebase/Supabase o un webhook de automatización).

> **Patrón alternativo: puente por automatización**
> Si la plataforma elegida no permite ocultar la clave o hacer la llamada directa, se puede enviar los datos a un **webhook de Make o n8n**, que llama a Gemini y devuelve el resultado. La clave vive solo en la herramienta de automatización.

Unidad 5. Gestión de usuarios, autenticación y planes de suscripción

5.1 Autenticación y autorización

- **Autenticación:** comprobar quién es el usuario (correo y contraseña, inicio de sesión con Google).
- **Autorización:** decidir qué puede ver o hacer (sus propios datos, funciones de su plan).

5.2 Modelo de datos mínimo

Tipo de dato	Campos	Descripción
User (incorporado)	email, `plan`, `creditos`, `usos_hoy`, `fecha_ultimo_uso`, `stripe_customer_id`	Cuenta del usuario con su plan y saldo
Generation	`usuario`, `entrada`, `salida`, `tokens`, `fecha`	Cada resultado generado
Plan (conjunto de opciones)	Gratis, Pro	Niveles de servicio
Config	`modelo_id`, `instruccion_sistema`, `limite_gratis`, `limite_pro`	Parámetros editables sin tocar la lógica

5.3 Reglas de privacidad

En Bubble, las **reglas de privacidad** son la defensa real de los datos. Sin ellas, cualquier usuario podría consultar los datos de otros.

- **Generation:** visible solo si `Creator is Current User`.
- **User:** campos como `creditos` y `plan` visibles solo para el propio usuario y modificables solo desde flujos de backend.
- Probar siempre con **dos cuentas** distintas para verificar el aislamiento.

5.4 Flujos de acceso

Flujo	Qué debe ocurrir
Registro	Crear usuario con plan Gratis, créditos iniciales y `usos_hoy = 0`
Inicio de sesión	Autenticar y redirigir a la página de la aplicación
Recuperación de contraseña	Enviar enlace de restablecimiento al correo
Cierre de sesión	Cerrar sesión y volver a la página pública
Protección de páginas	Si el usuario no ha iniciado sesión, redirigir a login

5.5 Planes y límites

Característica	Gratis	Pro
Generaciones por día	5	100
Historial guardado	Últimas 10	Ilimitado
Exportar resultado	No	Sí
Soporte	Autoservicio	Prioritario

Los valores son un ejemplo: se definen según el costo real por generación y la cuota disponible (Unidades 9 y 10). La app consulta el plan del usuario antes de cada generación y aplica el límite correspondiente.

5.6 Protección de datos personales

- Recolectar solo los datos necesarios.
- Incluir **política de privacidad** y **términos de uso** accesibles desde el registro.
- Informar que los textos del usuario se envían a un proveedor de IA externo.
- En el plan gratuito de Gemini, **no procesar datos sensibles ni de terceros**.

Práctica P1. Prototipo de app de IA en una plataforma no-code

Campo	Detalle
Módulo / unidades	8.1 · Unidades 1, 2 y 3
Duración	2,5 horas
Herramienta	Bubble (principal), Glide o FlutterFlow (alternativas)
Modalidad	Individual, guiada
Resultado	Prototipo navegable con formulario, resultado simulado y base de datos

Escenario

Una agencia de marketing quiere ofrecer a sus clientes un **generador de respuestas a reseñas** para negocios locales. Antes de invertir en la conexión con IA, se necesita validar la interfaz y el flujo con un prototipo que simule la respuesta.

Objetivos de aprendizaje

- Elegir y justificar la plataforma no-code.
- Diseñar las pantallas y el modelo de datos de un wrapper.
- Construir un flujo funcional con resultado simulado.

Desarrollo paso a paso (Bubble)

Paso 1. Definir el producto (20 min)

- Completar una ficha: usuario objetivo, problema, entrada, salida y diferenciador.
- Dibujar en papel o en una herramienta de diseño las 3 pantallas: **Inicio**, **Aplicación** e **Historial**.

Paso 2. Crear el modelo de datos (20 min)

- Crear el tipo de dato `Generation` con los campos: `usuario` (User), `resena` (texto), `tono` (texto), `respuesta` (texto), `fecha` (fecha).
- Crear el conjunto de opciones `Tono` con valores: Formal, Cercano, Empático.

Paso 3. Construir la pantalla de aplicación (40 min)

1. Agregar un campo de texto multilínea “Pega aquí la reseña” con límite de 600 caracteres.
2. Agregar un desplegable de **Tono** conectado al conjunto de opciones.
3. Agregar un selector de **calificación** (1 a 5).
4. Agregar el botón “Generar respuesta”.
5. Agregar un grupo de resultado oculto, con un texto dinámico y botones “Copiar” y “Regenerar”.

Paso 4. Programar el flujo con respuesta simulada (30 min)

En el workflow del botón, simular la respuesta con un texto fijo que incluya los datos del formulario:

Lógica del flujo simulado

```
Evento: Click en "Generar respuesta"
  Solo cuando: el campo de reseña no está vacío
  Acción 1: Crear una Generation (resena = campo de reseña, tono = desplegable, respuesta =
"Gracias por tu reseña de [calificación] estrellas. [Texto simulado en tono [tono]]")
  Acción 2: Mostrar el grupo de resultado con la respuesta creada
```

Paso 5. Pantalla de historial (20 min)

- Agregar una lista repetidora que muestre las Generations ordenadas por fecha descendente.
- Cada fila muestra reseña, tono y respuesta.

Paso 6. Prueba y documentación (20 min)

- Ejecutar el flujo con 5 reseñas distintas y registrar resultados.
- Capturar las 3 pantallas y el modelo de datos.
- Redactar la justificación de la plataforma (máximo 150 palabras).

Alternativas por plataforma

Plataforma	Cómo construir el prototipo
Glide	Tabla `Generaciones` + componente de formulario + acción “Agregar fila” con respuesta simulada + lista de historial.
FlutterFlow	Páginas con TextField, DropDown y Button; acción que agrega documento a la colección `generations` y muestra el resultado.

Entregables

- Enlace al prototipo funcional.
- Capturas de las 3 pantallas y del modelo de datos.
- Ficha del producto y justificación de la plataforma.
- Bitácora con 5 pruebas.

Rúbrica de evaluación

Criterio	Descripción	Puntos
Definición del producto	Ficha clara con usuario, problema, entrada y salida	20
Modelo de datos	Tipos de dato y campos coherentes	20
Interfaz	Formulario completo, estados y resultado visible	25

Criterio	Descripción	Puntos
Flujo funcional	La generación simulada se guarda y se muestra	25
Documentación	Capturas, bitácora y justificación	10
Total		100

Errores comunes

- Construir muchas pantallas antes de validar el flujo principal.
- Usar campos de texto libre donde un desplegable evitaría ambigüedad.
- Olvidar el estado vacío del historial.

Práctica P2. Conexión de la app a la API de Gemini

Campo	Detalle
Módulo / unidades	8.1 · Unidad 4
Duración	2,5 horas
Requisitos	Prototipo de P1 terminado; clave de API de Google AI Studio
Modalidad	Individual, guiada
Resultado	La app genera respuestas reales con Gemini, con manejo de errores y límite de uso

Escenario

El prototipo de P1 ya fue aprobado por el cliente. Ahora debe reemplazarse la respuesta simulada por una respuesta real generada por Gemini, cuidando la seguridad de la clave y los límites del plan gratuito.

Objetivos de aprendizaje

- Configurar el conector de API con la clave como parámetro privado.
- Redactar y probar una instrucción del sistema especializada.
- Manejar errores y cuotas con mensajes amigables.

Desarrollo paso a paso

Paso 1. Preparar credenciales y límites (15 min)

- Crear la clave en Google AI Studio y copiar el nombre del modelo Flash vigente.
- Anotar en la bitácora los límites del proyecto (peticiones por minuto y por día).

Paso 2. Configurar el API Connector (30 min)

Seguir los pasos de la Unidad 4.9. Cuerpo sugerido para la llamada `generar`:

Cuerpo con parámetros

```
{
  "systemInstruction": { "parts": [{ "text": "<sistema>" }] },
  "contents": [ { "role": "user", "parts": [{ "text": "<entrada>" }] } ],
  "generationConfig": { "temperature": 0.6, "maxOutputTokens": 800 }
}
```

Paso 3. Redactar la instrucción del sistema (30 min)

Instrucción del sistema (ejemplo)

```
Eres un especialista en atención al cliente de negocios locales de habla hispana.
Tu tarea: redactar la respuesta pública del negocio a una reseña.
Reglas:
- Responde en español, en máximo 90 palabras.
- Adapta el tono al solicitado (Formal, Cercano o Empático).
- Si la calificación es 1 o 2, reconoce el problema y ofrece contacto privado, sin
discutir.
- No inventes datos del negocio (horarios, precios, promociones).
- No incluyas saludos de despedida genéricos como "Atentamente, Equipo".
- Devuelve únicamente el texto de la respuesta.
```

Paso 4. Conectar el workflow (30 min)

Lógica del flujo real

```
Evento: Click en "Generar respuesta"
  Solo cuando: reseña no vacía Y usos_hoy del usuario < límite del plan
  Acción 1: Llamar Gemini - generar
            sistema = instrucción del sistema (desde Config)
            entrada = "Calificación: [n] estrellas. Tono: [tono]. Reseña: [texto]"
  Acción 2 (solo si no hay error): Crear Generation con la respuesta del paso 1
  Acción 3 (solo si no hay error): usos_hoy = usos_hoy + 1
  Acción 4 (solo si hay error): Mostrar mensaje amable según el código de error
```

Paso 5. Pruebas (30 min)

Caso de prueba	Resultado esperado
Reseña positiva de 5 estrellas	Respuesta cordial y breve
Reseña de 1 estrella con queja	Reconocimiento y oferta de contacto privado
Reseña vacía	El botón no ejecuta y se muestra aviso
Reseña en inglés	Respuesta en español, según la instrucción
Intento de manipulación (“ignora tus instrucciones…”)	La respuesta mantiene el rol; se documenta el comportamiento
Superar el límite diario del usuario	Mensaje de límite alcanzado, sin llamar a la API

Paso 6. Manejo del error 429 (15 min)

Provocar (o simular) una cuota agotada y verificar que la app muestra: “Estamos recibiendo muchas solicitudes. Intenta de nuevo en un minuto.” en lugar de un error técnico.

Entregables

- Capturas del API Connector con la clave **oculta** y la llamada inicializada.
- Documento con la instrucción del sistema y la tabla de 6 pruebas.

- Captura de la consola de límites del proyecto.
- Demostración de la app generando una respuesta real.

Rúbrica de evaluación

Criterio	Descripción	Puntos
Seguridad de la clave	Parámetro privado, sin exposición en la interfaz	20
Conexión funcional	La llamada devuelve y muestra texto real	25
Instrucción del sistema	Rol, formato, tono y límites bien definidos	20
Manejo de errores y cuota	Mensajes amigables y límite por usuario	20
Pruebas y documentación	Tabla de pruebas y capturas	15
Total		100

Errores comunes

- Dejar la clave sin marcar como privada.
- No usar formato JSON-safe y recibir error 400 con textos que contienen comillas o saltos de línea.
- Olvidar actualizar `MODEL_ID` cuando el modelo es retirado (error 404).
- No limitar `maxOutputTokens` y obtener respuestas largas y costosas.

Práctica P3. Integración de un sistema de autenticación de usuarios

Campo	Detalle
Módulo / unidades	8.1 · Unidad 5
Duración	2,5 horas
Requisitos	App de P2 funcionando
Modalidad	Individual, guiada
Resultado	Registro, login, recuperación de contraseña, datos privados por usuario y límites por plan

Escenario

La app ya genera respuestas, pero cualquier persona puede usarla y ver todas las generaciones. Se debe proteger el acceso y garantizar que cada usuario vea únicamente su propio historial, aplicando límites distintos según su plan.

Objetivos de aprendizaje

- Implementar registro, inicio y cierre de sesión.
- Configurar reglas de privacidad que aíslen los datos de cada usuario.
- Aplicar límites por plan antes de llamar a la API.

Desarrollo paso a paso

Paso 1. Ampliar el usuario (15 min)

- Agregar al tipo User: `plan` (conjunto de opciones Plan), `creditos`, `usos_hoy`, `fecha_ultimo_uso`.
- Crear el tipo `Config` con `limite_gratis` = 5 y `limite_pro` = 100.

Paso 2. Páginas de acceso (40 min)

1. Crear la página **login** con campos de correo y contraseña y botón “Iniciar sesión”.
2. Crear la página **registro** con correo, contraseña y casilla de aceptación de términos.
3. Crear la opción “Olvidé mi contraseña” con el flujo de restablecimiento.
4. Opcional: agregar inicio de sesión con Google mediante el plugin correspondiente.

Paso 3. Flujos de autenticación (30 min)

Flujos principales

```
Registro:  Registrar usuario (correo, contraseña)
           → establecer plan = Gratis, usos_hoy = 0
           → ir a la página "app"
Login:     Iniciar sesión → ir a la página "app"
Logout:    Cerrar sesión → ir a "inicio"
Protección: Al cargar la página "app", si el usuario no ha iniciado sesión → ir a "login"
```

Paso 4. Reglas de privacidad (30 min)

- Para `Generation`: "Cuando el creador es el usuario actual" → puede ver todos los campos y buscar en listas.
- Para `User`: el resto de usuarios no puede ver campos de otros; `plan` y `creditos` no se modifican desde el navegador.
- Crear la regla por defecto "Todos los demás": sin acceso.

Paso 5. Límite por plan y reinicio diario (25 min)

Lógica del límite

```
Antes de llamar a Gemini:
  Si fecha_ultimo_uso ≠ hoy → usos_hoy = 0 y fecha_ultimo_uso = hoy
  Límite = (plan = Pro) ? limite_pro : limite_gratis
  Si usos_hoy >= Límite → mostrar "Alcanzaste tu límite diario" y no llamar a la API
```

Paso 6. Prueba con dos cuentas (30 min)

- Registrar la **Cuenta A** y la **Cuenta B** con correos distintos.
- Generar 3 respuestas con A y 2 con B.
- Verificar que el historial de cada cuenta muestra solo sus propios registros.
- Intentar entrar a la página "app" sin sesión y comprobar la redirección.
- Superar el límite gratuito con la Cuenta A y confirmar el bloqueo.

Entregables

- Capturas de las páginas de registro, login y recuperación.
- Captura de las reglas de privacidad.
- Video corto o capturas de la prueba con dos cuentas.
- Tabla de pruebas con resultados.

Rúbrica de evaluación

Criterio	Descripción	Puntos
Registro y acceso	Registro, login, logout y recuperación operativos	25
Reglas de privacidad	Aislamiento comprobado entre usuarios	30

Criterio	Descripción	Puntos
Límites por plan	Reinicio diario y bloqueo previo a la llamada	20
Protección de páginas	Redirección sin sesión	10
Pruebas y documentación	Evidencia con dos cuentas	15
Total		100

Errores comunes

- Dejar las reglas de privacidad con acceso total “para probar” y olvidarlo al publicar.
- Confiar solo en ocultar elementos de la interfaz: la protección real son las reglas de datos.
- No reiniciar `usos_hoy` al cambiar de día.

MÓDULO 8.2: Despliegue y Monetización de Apps de IA

Duración: 10 horas | **Unidades:** 6 a 10 | **Prácticas:** P4 y P5

Unidad 6. Integración de pagos (Stripe) en una app de IA no-code

6.1 Conceptos de Stripe

Concepto	Qué es	Uso en el wrapper
Product	Lo que se vende	“Plan Pro”, “Paquete de 50 créditos”
Price	Precio y periodicidad de un producto	9 USD/mes o 5 USD por única vez
Checkout Session	Página de pago alojada por Stripe	El usuario paga sin que la app maneje tarjetas
Payment Link	Enlace de pago creado sin código	Alternativa simple para vender un plan
Subscription	Cobro recurrente	Plan Pro mensual
Customer	Cliente registrado en Stripe	Se enlaza con el usuario de la app
Webhook	Aviso automático de Stripe a la app	Confirma el pago y activa el plan
Customer Portal	Portal para gestionar suscripción	Cancelar, cambiar tarjeta, ver facturas

Disponibilidad por país

Stripe **no está disponible en todos los países** para abrir una cuenta que cobre. Antes del curso, cada participante debe verificar si puede crear una cuenta en su país. Si no es posible, las alternativas incluyen plataformas que actúan como comerciante registrado (por ejemplo, Paddle o Lemon Squeezy), PayPal, Mercado Pago u otras pasarelas locales. La lógica de la práctica (checkout → webhook → actualizar plan) es idéntica en todas.

6.2 Flujo de pago de una app de IA

Flujo de pago

```
1. El usuario pulsa "Mejorar a Pro" en la app.
2. La app lo envía a Stripe Checkout (con su identificador de usuario).
3. El usuario paga en la página segura de Stripe.
4. Stripe redirige al usuario a la página "pago exitoso".
5. Stripe envía un WEBHOOK a la app: "pago completado".
6. La app (flujo de backend) verifica el evento y actualiza el plan o los créditos.
7. El usuario ve su nuevo plan activo.
```

Principio clave

La redirección a la página “pago exitoso” **no prueba el pago**. La activación del plan debe depender del **webhook** (o de una consulta a Stripe), nunca de que el usuario llegue a una página.

6.3 Dos formas de integrarlo en no-code

Opción	Cómo funciona	Cuándo usarla
Payment Link + webhook	Se crea el enlace en el panel de Stripe; la app lo abre con el identificador del usuario; un webhook activa el plan	La más rápida; ideal para la práctica
Plugin de Stripe del builder	Checkout y suscripciones configuradas dentro del builder	Si el builder ofrece plugin oficial vigente
API de Stripe vía API Connector	Crear Checkout Sessions por API con la clave secreta privada	Mayor control (precios dinámicos, metadatos)

6.4 Webhooks en el builder

Un webhook es una URL de la app que Stripe llama cuando ocurre un evento. En Bubble se implementa con un **flujo de API (backend workflow)** expuesto como endpoint, función que puede requerir un plan de pago del builder.

Evento de Stripe	Acción en la app
`checkout.session.completed`	Activar plan Pro o sumar créditos al usuario identificado
`invoice.paid`	Renovar el periodo del plan
`invoice.payment_failed`	Marcar el pago como fallido y avisar al usuario
`customer.subscription.deleted`	Volver al plan Gratis

6.5 Seguridad de los pagos

- La app **nunca** recibe ni guarda números de tarjeta: los maneja Stripe.
- La **clave secreta** de Stripe va como parámetro privado, igual que la clave de Gemini.
- Verificar cada webhook: Stripe firma sus eventos; si el builder no permite validar la firma, consultar a Stripe por API el estado de la sesión antes de activar el plan.
- Hacer los flujos **idempotentes**: si el mismo evento llega dos veces, no debe sumar créditos dos veces (guardar el identificador del evento procesado).
- Probar siempre en **modo de prueba** antes de activar el modo real.

6.6 Datos de prueba

Caso	Tarjeta de prueba
Pago exitoso	`4242 4242 4242 4242` (cualquier fecha futura y cualquier CVC)
Pago rechazado	`4000 0000 0000 0002`

Unidad 7. Despliegue y hosting de aplicaciones no-code

7.1 Entornos de trabajo

Entorno	Propósito	Reglas
Desarrollo	Construir y probar cambios	Claves de prueba (Gemini de pruebas, Stripe en modo test)
Producción (live)	Versión que usan los clientes	Claves reales, reglas de privacidad verificadas

La mayoría de builders mantiene una versión de desarrollo y otra pública. Los cambios se **despliegan** a producción solo después de probarlos. Las claves de cada entorno deben ser distintas.

7.2 Hosting, dominio y certificado

- **Hosting:** lo provee la plataforma no-code (no se administra un servidor propio).
- **Dominio propio:** mejora la confianza y la marca; en muchos planes requiere pago.
- **SSL (HTTPS):** necesario para pagos y para proteger la sesión; los builders lo incluyen.
- **Correos transaccionales:** configurar remitente propio para recuperación de contraseña y avisos.

7.3 Páginas legales y cumplimiento básico

- **Términos de servicio:** reglas de uso, límites de responsabilidad, política de reembolsos.
- **Política de privacidad:** qué datos se recogen, para qué, y que se procesan con un proveedor de IA externo.
- **Aviso de contenido generado por IA** visible en la aplicación.
- **Consentimiento de cookies** si se usa analítica.

7.4 Lista de verificación previa al lanzamiento

Área	Verificación	✔
Seguridad	Claves privadas; reglas de privacidad probadas con dos cuentas	☐
IA	Modelo vigente (`MODEL_ID`); límite diario por usuario; mensajes de error	☐
Pagos	Webhook probado; idempotencia; modo real activado solo al final	☐
Costos	Facturación de Gemini activada si habrá uso comercial; alertas de gasto	☐
Legal	Términos, privacidad y aviso de IA publicados	☐
Calidad	Pruebas en móvil y escritorio; estados vacíos y de carga	☐
Operación	Correo de soporte; respaldo de datos; plan de contingencia si la API falla	☐

7.5 De gratuito a producción con Gemini

> **Transición de la capa de IA**
> El plan gratuito de Gemini es ideal para aprender y prototipar. Para ofrecer el servicio a clientes que pagan, se debe: (1) revisar los términos vigentes sobre uso comercial y datos; (2) **activar la facturación** del proyecto en Google; (3) crear una **clave nueva de producción**; (4) configurar **alertas de presupuesto**; y (5) volver a medir el costo por generación con los precios del nivel de pago.

Unidad 8. Pruebas de usabilidad y mejora iterativa

8.1 ¿Por qué probar con usuarios reales?

El creador conoce su app demasiado bien. Observar a una persona nueva usándola revela fricciones que el autor no ve. En productos de IA además hay que evaluar la **calidad percibida del resultado**, no solo la facilidad de uso.

8.2 Plan de prueba de usabilidad

1. **Definir tareas concretas**, por ejemplo: “Regístrate y genera una respuesta a esta reseña”.
2. **Reclutar a 5 personas** del público objetivo; con 5 participantes suelen aparecer la mayoría de los problemas principales.
3. **Observar sin ayudar**: pedir que piensen en voz alta y anotar dónde dudan.
4. **Medir**: tareas completadas, tiempo, errores y satisfacción.
5. **Cerrar con preguntas**: ¿qué fue confuso? ¿pagarías por esto? ¿qué falta?

8.3 Métricas de usabilidad

Métrica	Cómo se mide	Meta orientativa
Tasa de éxito de la tarea	Participantes que la completan sin ayuda	≥ 80 %
Tiempo hasta el primer resultado	Desde que abre la app hasta ver una respuesta	< 3 minutos
Errores	Número de equivocaciones por tarea	Tender a 0
Satisfacción (escala SUS)	Cuestionario estándar de 10 preguntas	≥ 68 puntos
Calidad del resultado	Valoración del usuario de 1 a 5	≥ 4

8.4 Ciclo de mejora iterativa

Ciclo de mejora

```
Observar → Identificar el problema → Formular hipótesis → Cambiar una cosa → Medir →
Decidir
Ejemplo:
  Problema: 3 de 5 usuarios no entienden qué escribir en el campo de reseña.
  Hipótesis: un ejemplo precargado reducirá la confusión.
  Cambio: botón "Probar con un ejemplo".
  Medición: % que genera su primer resultado en < 3 minutos.
```

8.5 Priorización

	Esfuerzo bajo	Esfuerzo alto
Impacto alto	Hacer primero	Planificar
Impacto bajo	Hacer si hay tiempo	Descartar

8.6 Mejoras específicas de productos de IA

- Ajustar la instrucción del sistema según los fallos observados y guardar **versiones** del prompt.
- Añadir botón “Regenerar” y valoración / del resultado para recoger datos de calidad.
- Mantener un conjunto fijo de **entradas de prueba** para comprobar que un cambio de prompt no empeora casos anteriores.

Unidad 9. Métricas de uso y analítica de producto

9.1 Marco de métricas

Etapa	Pregunta	Métricas
Adquisición	¿Cómo llegan?	Visitas, registros, tasa de visita → registro
Activación	¿Viven el valor del producto?	% que genera su primer resultado
Retención	¿Regresan?	Usuarios activos semanales; regreso a los 7 días
Ingresos	¿Pagan?	Conversión gratis → pago, MRR, ARPU
Recomendación	¿Lo recomiendan?	Invitaciones, valoraciones, comentarios

9.2 Métricas específicas de apps de IA

Métrica	Fórmula / fuente	Para qué sirve
Generaciones por usuario	Total de Generations ÷ usuarios activos	Mide uso y presión sobre la cuota
Tokens por generación	`usageMetadata` de la respuesta	Base del costo
Costo por generación	Ver Unidad 10	Define precios y márgenes

Métrica	Fórmula / fuente	Para qué sirve
Tasa de errores de IA	Llamadas fallidas ÷ llamadas totales	Detecta problemas de cuota o de prompt
Satisfacción del resultado	% de valoraciones	Mide la calidad del prompt

9.3 Métricas de negocio

Métrica	Cálculo
MRR (ingreso mensual recurrente)	Suscriptores activos × precio mensual
ARPU	Ingresos del periodo ÷ usuarios activos
Conversión gratis → pago	Usuarios que pagan ÷ usuarios registrados
Churn mensual	Suscriptores que cancelan ÷ suscriptores al inicio del mes
Margen bruto	(Ingresos − costo de IA − comisiones de pago) ÷ ingresos

9.4 Cómo medir en no-code

- **Panel interno:** una página de administrador en el propio builder, con búsquedas y totales sobre las tablas `User` y `Generation`.
- **Analítica web:** Google Analytics 4 o herramientas de producto como PostHog o Mixpanel, enviando eventos (registro, primera generación, compra).
- **Stripe:** su panel muestra ingresos, suscripciones activas y cancelaciones.
- **Cookies y privacidad:** informar y obtener consentimiento cuando corresponda.

9.5 Tablero mínimo recomendado

Indicador	Frecuencia
Registros nuevos	Diaria
Activación (primer resultado)	Semanal
Generaciones totales y por usuario	Diaria
Errores de la API (429, 5xx)	Diaria
Conversión a pago y MRR	Semanal
Costo de IA acumulado del mes	Semanal

Unidad 10. Estrategias de monetización: SaaS, freemium y créditos

10.1 Modelos de monetización

Modelo	Cómo funciona	Ventajas	Riesgos
SaaS (suscripción)	Pago mensual o anual con límites por plan	Ingresos predecibles	Exige retención constante
Freemium	Plan gratuito limitado + plan de pago	Atrae usuarios con facilidad	El plan gratuito genera costo de IA
Créditos	Paquetes prepagados; cada uso descuenta	Costo alineado con el consumo	Menos ingreso recurrente
Híbrido	Suscripción con créditos incluidos + recarga	Flexible	Más complejo de explicar

10.2 Cálculo del costo por generación

Fórmula

$$\text{costo por generación} = \frac{(\text{tokens de entrada} \times \text{precio de entrada}) + (\text{tokens de salida} \times \text{precio de salida})}{1.000.000}$$

Los tokens se leen en `usageMetadata` de cada respuesta y los precios se toman de la página oficial de precios de Gemini. Ejemplo con **valores ilustrativos** (deben reemplazarse por los precios vigentes):

Dato	Valor ilustrativo
Tokens de entrada por generación	700
Tokens de salida por generación	300
Precio de entrada (por millón de tokens)	0,30 USD
Precio de salida (por millón de tokens)	2,50 USD
Costo por generación	(700 × 0,30 + 300 × 2,50) ÷ 1.000.000 ≈ 0,00096 USD
Uso de un usuario Pro (100 por día × 30 días)	3.000 generaciones ≈ 2,88 USD al mes

> **Importante**
> En modelos con razonamiento, los tokens de “pensamiento” también se facturan como salida. Medir siempre con datos reales de `usageMetadata`, no solo con estimaciones.

10.3 Fijación de precios

1. **Calcular el costo** por generación y el consumo esperado por plan.

2. **Sumar comisiones** de la pasarela de pago (un porcentaje más una tarifa fija por transacción; verificar la tarifa vigente en el país).
3. **Definir el margen objetivo**, por ejemplo 70 % de margen bruto.
4. **Comparar con el valor** que recibe el cliente y con la competencia.
5. **Probar** tres precios con usuarios reales antes de fijar uno.

> **Regla de oro**
> **Precio por usuario > costo de IA por usuario + comisiones + costo de la plataforma**, con un margen que permita crecer.

10.4 Diseño de los planes

Elemento	Gratis	Pro	Paquete de créditos
Precio	0	Mensual	Pago único
Uso	Tope diario bajo	Tope diario alto	Hasta agotar créditos
Objetivo	Probar el valor	Uso frecuente	Uso ocasional
Riesgo a controlar	Costo por usuarios que no pagan	Abuso del tope	Créditos sin vencimiento

10.5 Errores de monetización

- Plan gratuito demasiado generoso: agota la cuota y los ingresos no cubren el costo.
- No medir el costo real por usuario antes de fijar precios.
- Cobrar sin entregar una mejora clara respecto al plan gratuito.
- No tener política de reembolsos ni forma de cancelar.

Práctica P4. Integración de pagos con Stripe en la aplicación

Campo	Detalle
Módulo / unidades	8.2 · Unidad 6
Duración	2,5 horas
Requisitos	App con autenticación (P3); cuenta de Stripe (o alternativa) en modo de prueba
Modalidad	Individual, guiada
Resultado	Un usuario paga en modo de prueba y la app actualiza su plan automáticamente

Escenario

La app ya tiene usuarios y límites por plan. Ahora se debe permitir que un usuario del plan Gratis compre el **plan Pro** y que su cuenta se actualice sin intervención manual.

Objetivos de aprendizaje

- Crear productos y precios en Stripe.
- Conectar el botón “Mejorar a Pro” con el pago.
- Procesar un webhook para activar el plan de forma segura.

Desarrollo paso a paso

Paso 1. Configurar Stripe en modo de prueba (20 min)

1. Verificar que el interruptor del panel indique **modo de prueba**.
2. Crear el producto **Plan Pro** con un precio mensual (por ejemplo, 9 USD).
3. Opcional: crear el producto **Paquete de 50 créditos** con pago único.
4. Crear un **Payment Link** para el plan Pro y copiar su URL.

Paso 2. Botón de compra en la app (20 min)

- Crear la página **planes** con la tabla comparativa Gratis/Pro de la Unidad 5.5.
- El botón “Mejorar a Pro” abre el enlace de pago agregando el identificador del usuario y su correo (`client_reference_id` y correo prellenado como parámetros de la URL).

Estructura del enlace de pago

```
https://buy.stripe.com/ENLACE_DE_PRUEBA?client_reference_id=<ID del usuario
actual>&prefilled_email=<correo del usuario actual>
```

Paso 3. Endpoint de webhook en la app (35 min)

1. Crear un **flujo de API (backend workflow)** llamado `stripe_webhook` y exponerlo como endpoint público.
2. Definir los parámetros que recibirá del evento (tipo de evento, identificador del evento, referencia del usuario, estado del pago).
3. Copiar la URL del endpoint.
4. En Stripe, ir a **Desarrolladores → Webhooks**, agregar el endpoint y suscribirlo a `checkout.session.completed`, `invoice.paid`, `invoice.payment_failed` y `customer.subscription.deleted`.

Paso 4. Lógica del webhook (35 min)

Lógica del webhook

```
Al recibir un evento:
  1. Si el id del evento ya existe en la tabla "EventoProcesado" → terminar (idempotencia)
  2. Verificar el pago: consultar a Stripe por API el estado de la sesión
     (o validar la firma si el builder lo permite)
  3. Si tipo = checkout.session.completed y estado = pagado:
        buscar usuario por client_reference_id
        plan = Pro ; guardar stripe_customer_id
  4. Si tipo = customer.subscription.deleted:
        buscar usuario por stripe_customer_id → plan = Gratis
  5. Guardar el id del evento en "EventoProcesado"
```

Paso 5. Prueba del pago (25 min)

- Iniciar sesión con la Cuenta A (plan Gratis) y pulsar “Mejorar a Pro”.
- Pagar con la tarjeta de prueba `4242 4242 4242 4242`.
- Verificar en el panel de Stripe que el pago aparece en modo de prueba.
- Verificar que el plan del usuario cambió a Pro y que su límite diario subió.
- Repetir con la tarjeta rechazada `4000 0000 0000 0002` y comprobar que el plan **no** cambia.

Paso 6. Gestión de la suscripción (15 min)

Activar el **Customer Portal** de Stripe y agregar el botón “Gestionar suscripción” que lleve al portal. Cancelar la suscripción de prueba y confirmar que el usuario vuelve al plan Gratis.

Alternativa si Stripe no está disponible

Si el participante no puede crear una cuenta de Stripe en su país, debe realizar la práctica con una pasarela alternativa (por ejemplo, PayPal, Mercado Pago o un comerciante registrado como Paddle/Lemon Squeezy) en su modo de pruebas. La estructura se mantiene: enlace de pago con referencia del usuario, notificación automática (webhook/IPN) y actualización del plan.

Entregables

- Capturas del producto, precio y Payment Link en Stripe (modo de prueba).
- Captura del flujo de API con la lógica del webhook.

- Evidencia del pago exitoso y del cambio de plan.
- Evidencia del pago rechazado sin cambio de plan.

Rúbrica de evaluación

Criterio	Descripción	Puntos
Configuración de pagos	Producto, precio y enlace correctos en modo de prueba	20
Flujo de compra	Botón, identificación del usuario y redirecciones	20
Webhook y activación del plan	Plan Pro activado solo tras confirmar el pago	30
Seguridad e idempotencia	Verificación del evento y sin duplicados	15
Pruebas y documentación	Pago exitoso, rechazado y cancelación	15
Total		100

Errores comunes

- Activar el plan al llegar a la página “pago exitoso” en lugar de esperar el webhook.
- Mezclar claves de modo de prueba y modo real.
- No pasar la referencia del usuario y no poder saber a quién activar.
- Procesar dos veces el mismo evento y sumar créditos duplicados.

Práctica P5 (Proyecto final). Wrapper de IA funcional, desplegado y monetizable

Campo	Detalle
Módulo / unidades	8.1 y 8.2 · Unidades 1 a 10
Duración	2 horas en clase + trabajo autónomo
Modalidad	Individual o en parejas
Resultado	Aplicación en vivo con Gemini, autenticación, pagos, analítica y modelo de monetización
Evaluación	Presentación de 10 minutos + demostración + informe

Escenario

Cada participante lanza un **producto mínimo viable (MVP)** de IA para un nicho real. La aplicación debe poder recibir usuarios, generar resultados útiles con Gemini, cobrar y ser medida.

Requisitos del proyecto

Componente	Requisito mínimo
Nicho y problema	Usuario objetivo y problema concreto documentados
Interfaz	Formulario inteligente, resultado con copiar/regenerar, historial
IA	Conexión a Gemini con instrucción del sistema especializada, clave privada y manejo de errores
Usuarios	Registro, login, recuperación y datos privados por usuario
Planes	Al menos dos planes con límites distintos
Pagos	Flujo de compra completo en modo de prueba, con webhook
Despliegue	Versión en producción accesible por enlace; páginas legales y aviso de IA
Analítica	Panel con al menos 5 indicadores
Monetización	Análisis de costos y modelo de precios justificado

Ideas de proyecto

Proyecto	Usuario objetivo
Generador de propuestas comerciales	Freelancers y agencias

Proyecto	Usuario objetivo
Asistente de respuestas a reseñas	Restaurantes, tiendas y hoteles locales
Creador de planes de clase	Docentes
Redactor de fichas de producto	Tiendas en línea
Resumidor de contratos y documentos	Pymes y asesores
Preparador de entrevistas de trabajo	Buscadores de empleo

Plan de trabajo sugerido

Fase	Actividades	Entrega parcial
1. Definición	Nicho, problema, usuario, propuesta de valor, boceto de pantallas	Ficha del producto
2. Construcción	Interfaz, datos, conexión a Gemini, instrucción del sistema, pruebas de prompt	App funcional en desarrollo
3. Usuarios y pagos	Autenticación, planes, límites, Stripe y webhook	Flujo de compra probado
4. Validación	Prueba de usabilidad con 5 personas; mejoras priorizadas	Informe de usabilidad
5. Lanzamiento	Despliegue, páginas legales, analítica, análisis de costos	App en producción
6. Presentación	Demostración en vivo y defensa del modelo de negocio	Presentación

Entregables finales

1. **Enlace a la aplicación desplegada** y credenciales de una cuenta de prueba para el evaluador.
2. **Ficha del producto:** nicho, problema, propuesta de valor y diferenciador.
3. **Documento de la instrucción del sistema** con versiones y resultados de las pruebas.
4. **Informe de usabilidad:** plan, resultados de 5 usuarios, hallazgos y mejoras realizadas.
5. **Análisis de costos y precios:** tokens medios, costo por generación, diseño de planes, margen y punto de equilibrio.
6. **Tablero de métricas** con capturas de al menos 5 indicadores.
7. **Presentación** de 10 minutos y demostración en vivo.

Plantilla del análisis de costos

Concepto	Valor	Fuente

Concepto	Valor	Fuente
Tokens de entrada promedio		usageMetadata (10 pruebas)
Tokens de salida promedio		usageMetadata (10 pruebas)
Precio de entrada / salida por millón		Página oficial de precios de Gemini
Costo por generación		Cálculo (Unidad 10.2)
Generaciones esperadas por plan al mes		Estimación
Costo de IA por usuario y plan		Cálculo
Comisión de la pasarela de pago		Tarifa vigente
Costo mensual de la plataforma no-code		Plan contratado
Precio propuesto del plan Pro		Decisión justificada
Margen bruto por usuario Pro		Cálculo
Punto de equilibrio (usuarios Pro necesarios)		Costos fijos ÷ margen por usuario

Rúbrica de evaluación

Criterio	Descripción	Puntos
Propuesta de valor	Nicho claro, problema real y diferenciación frente a un chat genérico	10
Funcionalidad de IA	Calidad del prompt, formato de salida y manejo de errores	15
Usuarios y seguridad	Autenticación, privacidad de datos y clave protegida	15
Pagos	Flujo de compra y activación de plan verificados	15
Despliegue y legal	App accesible, páginas legales y aviso de IA	10
Usabilidad e iteración	Pruebas con usuarios y mejoras documentadas	10
Métricas	Tablero con indicadores relevantes	5
Costos y monetización	Análisis de costos, precios y márgenes coherentes	15
Presentación	Claridad, demostración en vivo y respuesta a preguntas	5
Total		100

Niveles de desempeño

Rango	Nivel	Descripción
90 – 100	Sobresaliente	Producto listo para vender, con datos que sustentan el modelo de negocio
75 – 89	Competente	Producto funcional con áreas menores por pulir
60 – 74	En desarrollo	Funciona parcialmente; faltan componentes de seguridad, pagos o análisis
< 60	Insuficiente	No cumple los requisitos mínimos

Anexo A. Plantillas de instrucción del sistema (systemInstruction)

Plantillas base para los proyectos de la P5. Se adaptan al nicho y se prueban con al menos 10 entradas.

A.1 Generador de propuestas comerciales

```
Eres un redactor comercial senior para freelancers y pequeñas agencias de habla hispana.
Redacta una propuesta comercial a partir de los datos recibidos.
Estructura obligatoria: 1) Resumen, 2) Alcance, 3) Inversión, 4) Plazo y siguientes pasos.
Máximo 300 palabras. Tono profesional y cercano.
No inventes cifras, nombres ni compromisos que no estén en los datos.
Si falta información esencial, indícala al final en una línea que empiece por "Falta:".
```

A.2 Planes de clase

```
Eres un diseñador instruccional con experiencia en educación de habla hispana.
Crea un plan de clase con: objetivo, materiales, inicio (10 %), desarrollo (70 %), cierre
(20 %) y evaluación.
Ajusta las actividades al nivel y duración indicados.
Usa lenguaje claro y actividades realizables con pocos recursos.
No incluyas contenido que requiera datos que el docente no proporcionó.
```

A.3 Fichas de producto para tiendas en línea

```
Eres un redactor de e-commerce especializado en SEO para tiendas de habla hispana.
Devuelve únicamente un JSON con las claves: "titulo" (máx. 70 caracteres), "descripcion"
(máx. 120 palabras), "beneficios" (lista de 5 frases cortas) y "palabras_clave" (lista de
8).
No inventes características técnicas que no estén en los datos del producto.
```

A.4 Resumidor de documentos

```
Eres un analista que resume documentos para personas sin formación técnica.
Entrega: 1) Resumen en máximo 120 palabras, 2) Puntos clave (5 viñetas), 3) Puntos de
atención o dudas.
No des asesoría legal, médica ni financiera.
Cierra siempre con: "Este resumen es orientativo y no sustituye la revisión de un
profesional."
```

Anexo B. Banco de pruebas para un wrapper de IA

Tipo de entrada	Ejemplo	Qué se evalúa
Caso normal	Datos completos y razonables	Calidad y formato
Datos incompletos	Falta un campo clave	Que no invente información
Entrada vacía o mínima	Un solo carácter	Validación y mensaje
Entrada muy larga	Texto de 5.000 caracteres	Límite de entrada y costo
Otro idioma	Texto en inglés	Cumplimiento del idioma de salida
Caracteres especiales	Comillas, saltos de línea, emojis	Ausencia de errores 400
Intento de manipulación	“Ignora tus instrucciones y…”	Que mantenga el rol y las reglas
Tema fuera de alcance	Pregunta no relacionada con el nicho	Rechazo amable o redirección
Contenido sensible	Solicitud inapropiada	Bloqueo y mensaje adecuado
Repetición	Misma entrada dos veces	Consistencia o uso de caché

Anexo C. Solución de problemas frecuentes

Síntoma	Causa probable	Solución
Error 400 al llamar a Gemini	JSON inválido por comillas o saltos de línea en las variables	Aplicar formato JSON-safe a cada parámetro de texto
Error 403	Clave incorrecta, revocada o header mal escrito	Verificar `x-goog-api-key` y regenerar la clave
Error 404	Modelo retirado o mal escrito	Actualizar `MODEL_ID` desde AI Studio
Error 429	Cuota del proyecto agotada	Reducir uso, añadir espera y reintento, revisar límites; considerar nivel de pago
Respuesta vacía o cortada	`maxOutputTokens` insuficiente o razonamiento consumiendo el cupo	Aumentar el tope o ajustar el razonamiento
Respuesta sin `candidates`	Bloqueo por filtros de seguridad	Mostrar mensaje y pedir reformular
Un usuario ve datos de otro	Reglas de privacidad ausentes o permisivas	Restringir por creador y probar con dos cuentas
El pago se hizo y el plan no cambió	Webhook no recibido o sin la referencia del usuario	Revisar el registro de eventos de Stripe y los parámetros del endpoint
Créditos duplicados	Evento procesado dos veces	Guardar y consultar el id del evento (idempotencia)
La app funciona en desarrollo, no en producción	Claves o configuración no replicadas	Revisar variables y reglas en el entorno live

Anexo D. Glosario

Término	Definición
API	Interfaz que permite a una aplicación comunicarse con otro servicio.
API Connector	Herramienta del builder para configurar llamadas HTTP a servicios externos.
Backend workflow	Flujo que se ejecuta en el servidor del builder, sin intervención del navegador.
Churn	Porcentaje de clientes que cancelan en un periodo.
Freemium	Modelo con plan gratuito limitado y plan de pago.
Idempotencia	Propiedad por la que repetir una operación no cambia el resultado.
MRR	Ingreso mensual recurrente.
MVP	Producto mínimo viable: versión básica para validar con usuarios reales.
No-code / Low-code	Desarrollo mediante interfaces visuales, con o sin fragmentos de código.
Prompt	Instrucción o texto que se envía a un modelo de lenguaje.
systemInstruction	Campo de Gemini donde se define el comportamiento permanente del asistente.
Token	Unidad de texto que procesa un modelo; base de la facturación.
Webhook	Aviso automático que un servicio envía a una URL cuando ocurre un evento.
Wrapper de IA	Aplicación que envuelve un modelo de IA con interfaz, instrucciones y datos propios.

Anexo E. Recursos y fuentes oficiales

Los precios, límites y funciones cambian con frecuencia. Antes de cada cohorte se recomienda revisar estas fuentes:

Tema	Dónde consultar
Clave y pruebas de Gemini	aistudio.google.com
Límites del proyecto	aistudio.google.com/rate-limit
Documentación y precios de Gemini API	ai.google.dev (secciones de documentación, límites de uso y precios)
Términos de la API de Gemini	ai.google.dev/gemini-api/terms
Bubble	Manual y foro oficial de Bubble (bubble.io)
Glide	Documentación oficial de Glide (glideapps.com)
FlutterFlow	Documentación oficial de FlutterFlow (flutterflow.io)
Stripe	Documentación de Stripe (stripe.com/docs): Checkout, Payment Links, webhooks y modo de prueba

Nota para el instructor

Este manual fue preparado con información vigente a octubre de 2026. Los nombres de modelos, los límites gratuitos y las funciones de los planes de las plataformas son los datos que con más probabilidad cambiarán; deben verificarse antes de dictar el curso.